

Game Rental System

ระบบเช่าเกม — เขียนด้วยภาษา Java

นาย นฤตม ณ นคร | รหัส 69319010006



ภาพรวมโปรเจกต์



Register Customer

สมัครสมาชิก



View Games

แสดงรายการเกม



Rent Game

เช่าเกม + คำนวณค่าเช่า



Return Game

คืนเกม + ตรวจสอบค่าปรับ



Keyboard Input

รับข้อมูลผ่าน Console

โครงสร้างระบบ



Layered Architecture

Main

จุดเริ่มต้นโปรแกรม

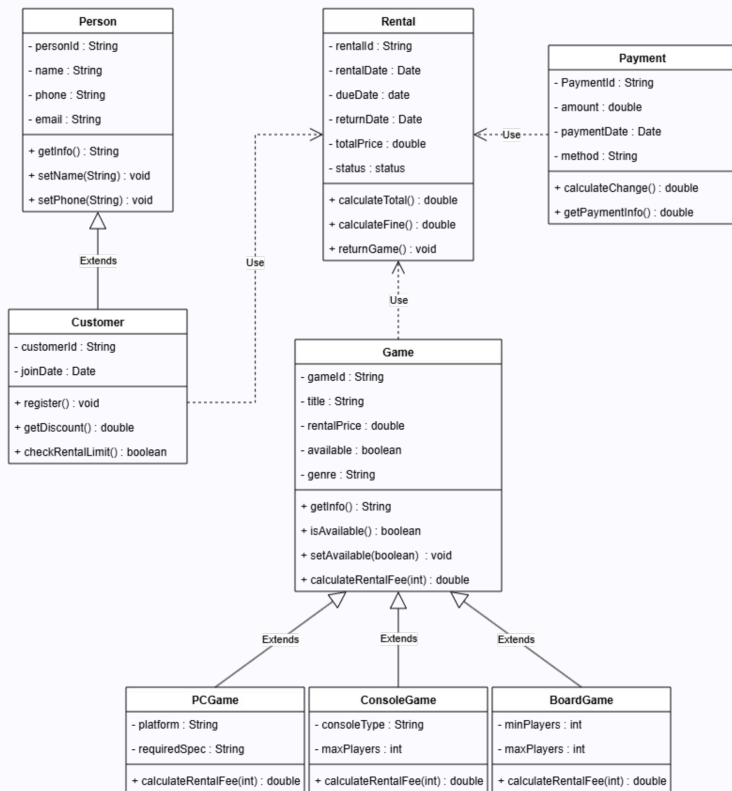
GameRentalSystem

จัดการการทำงานของระบบ

Model Layer

Person, Customer, Game
(PCGame, ConsoleGame, BoardGame)
Rental, Payment

ความสัมพันธ์ของ Class



การเชื่อมโยงระหว่าง Class

→ Inheritance

Customer extends Person

→ Inheritance

PCGame, ConsoleGame, BoardGame extends Game

→ Association

Rental มี Customer, Game, Payment

→ Polymorphism

จัดการเกมหลายประเภทผ่าน `ArrayList<Game>`

OOP #1 — Encapsulation

ควบคุมข้อมูลภายใน Object อย่างไร?

ใช้ **private fields** เพื่อไม่ให้แก้ไขโดยตรงจากภายนอก และใช้ **Method** ควบคุมการเปลี่ยนแปลงสถานะอย่างปลอดภัย

```
// Game.java
private boolean available;

public boolean rent() {
    if (!available) return false;

    available = false; // ✓ ควบคุมการเปลี่ยนแปลง
    return true;
}

public boolean returnGame() {
    if (available) return false;

    available = true; // ✓ คืนสถานะอย่างปลอดภัย
    return true;
}
```

private field

ไม่เปิดแก้ไขโดยตรง

Method

ควบคุมการเปลี่ยนแปลง

OOP #2 — Inheritance

```
// Person.java (Parent)
public abstract class Person {
    protected String name;
    protected String id;
}

// Customer.java (Child)
public class Customer extends Person {
    private List<Rental> rentalHistory;
    // เพิ่มความสามารถเฉพาะ
}

// Game.java (Parent)
public abstract class Game { ... }

public class PCGame extends Game { ... }
public class ConsoleGame extends Game { ... }
public class BoardGame extends Game { ... }
```

Person → Customer

สืบทอด name, id และเพิ่ม rentalHistory

ใช้ Inheritance ตรงไหน?

Class แม่เก็บคุณสมบัติร่วม เพื่อลดการเขียน Code ซ้ำ Class ลูกเพิ่มความสามารถเฉพาะของตัวเอง

Game → PCGame / ConsoleGame / BoardGame

สืบทอดคุณสมบัติเกมร่วม แต่ละประเภทมี Logic เฉพาะ

OOP #3 — Abstraction

ซ่อนรายละเอียด กำหนดพฤติกรรมร่วม

`abstract class Game` กำหนดโครงสร้างที่เกม**ทุกประเภท** ต้องมี แต่ละประเภทกำหนดรายละเอียดการคำนวณค่าเช่าเอง

// Game.java – Abstract Class

```
public abstract class Game {  
    protected String title;  
    protected double dailyRate;  
    protected boolean available;  
  
    public abstract double calculateRentalFee(int days);  
  
    public abstract void showInfo();  
}  
}
```

// PCGame.java – Concrete Class

```
public class PCGame extends Game {  
  
    @Override  
    public double calculateRentalFee(int days) {  
        return dailyRate * days * 1.0; // ✓ ราคาปกติ  
    }  
}
```

// BoardGame.java – Concrete Class

```
public class BoardGame extends Game {  
  
    @Override  
    public double calculateRentalFee(int days) {  
        return dailyRate * days * 0.8; // ✓ ลด 20%  
    }  
}
```

1 กำหนด Contract

บังคับให้ทุก Subclass ต้อง implement `calculateRentalFee()`

2 ซ่อน Implementation

ผู้ใช้งานไม่ต้องรู้ว่าแต่ละเกมคำนวณอย่างไร

OOP #4 — Polymorphism

```
// GameRentalSystem.java
ArrayList<Game> games = new ArrayList<>();

games.add(new PCGame("Elden Ring", 50));
games.add(new ConsoleGame("FIFA 25", 40));
games.add(new BoardGame("Catan", 30));

// ✓ Method เดียวกัน ทำงานต่างกัน
for (Game game : games) {
    game.showInfo();           // เรียก Method ของ
    Object จริง
    game.calculateRentalFee(3); // คำนวณตามประเภทจริง
}
```

Method เดียวกัน แต่ทำงานแตกต่างกัน

Java เลือก Method ของ Object จริงโดยอัตโนมัติ ณ Runtime ทำให้จัดการเกมหลายประเภทด้วย Code ชุดเดียว

PCGame

showInfo() แสดง Spec PC

ConsoleGame

showInfo() แสดง Platform

BoardGame

showInfo() แสดงจำนวนผู้เล่น

การทำงานของระบบ

1

สมัครสมาชิก

Register Customer → เลือก Customer

2

เลือกเกม

เลือก Game → กำหนดจำนวนวัน

3

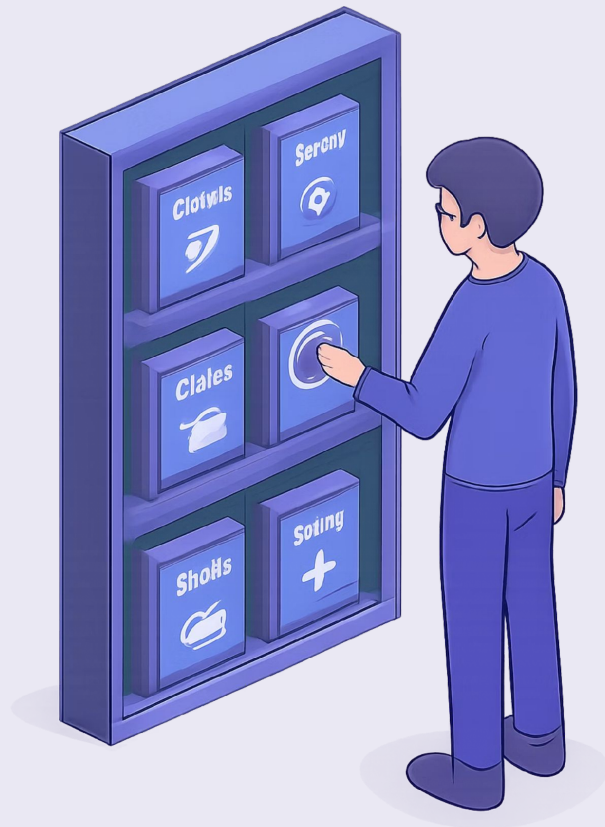
ชำระเงิน

เลือก Payment Method → สร้าง Rental

4

คืนเกม

Return Game → ตรวจสอบค่าปรับ



สรุป — OOP ในโปรแกรม

แนวคิด OOP	Class / Method ที่ใช้	ประโยชน์
Encapsulation	private available, rent(), returnGame()	ป้องกันการแก้ไขข้อมูลโดยตรง
Inheritance	Customer extends PersonPCGame/ConsoleGame/BoardGam e extends Game	ลด Code ซ้ำ แบ่งหน้าที่ชัดเจน
Abstraction	abstract class Person, GamecalculateRentalFee()	กำหนด Contract บังคับ Subclass
Polymorphism	ArrayList<Game>, showInfo(), calculateRentalFee()	Code ยืดหยุ่น รองรับประเภทใหม่ได้ง่าย

📌 ทั้ง 4 แนวคิด OOP ถูกนำมาใช้ร่วมกัน เพื่อให้ Code เป็นระบบ แยกหน้าที่ชัดเจน และขยายระบบได้ง่าย